Model-Based Rapid Prototyping of Tiered MFA for Cyber-Physical Systems in Contested Environments

Noah Stephenson
Department of Systems Engineering
United States Military Academy
West Point, New York, USA
noah.stephenson@westpoint.edu

Matthew Wolfe
Department of Systems Engineering
United States Military Academy
West Point, New York, USA
matthew.wolfe@westpoint.edu

***Abstract*—A SysML model governed the rapid prototyping of this tiered multi-factor authentication capability, and this paper reports what building against it surfaced. A tier is an operating mode of an authentication capability, defined by its required factors together with the scope it authorizes, the conditions it applies under, and what it records. What separates them is authorization scope, operating precondition, and accountability, so a tier is a joint constraint over several design points rather than a factor set. Where two tiers share authorization scope, the factor that only one of them requires protects nothing: an adversary holding a cloned badge and a known PIN reached all twenty protected resources by asserting the tier that does not require a fingerprint, the same twenty a legitimate Tier 1 subject reached. Requiring a minimum tier per resource contains the adversary, but no resource whose minimum sits above Tier 3 can also be reachable under degraded access. The model holds accountability fixed under degradation; the prototype did not, and all three tiers left approved access decisions unrecorded when the audit path failed mid-request. The operating tier is an input to the capability rather than something it selects, and nothing checks the asserted tier against the condition that prompted it. The experiment harness and its results are in a public repository. The prototype provides evidence of a potential implementation, not formal security validation.**

***Index Terms*—Model-Based Systems Engineering, SysML, multi-factor authentication, cyber-physical systems, degraded operation, accountability**

I. Introduction

Tactical cyber-physical systems (CPS) often lose connection within a network. Denied, degraded, intermittent, and limited (DDIL) conditions are the norm rather than the exception, and access control must keep working through them. Tiered multi-factor authentication (MFA) is one common response: define weaker operating modes that the system can fall back to when hardware or services degrade.

However, it is easy to specify a tier by factor sets, and the factor set is not enough: two policies may both require an RFID token and a PIN yet differ radically in scope and logging. The consequence is this: the tier is asserted by the requester rather than derived by the capability, so an adversary who holds a cloned badge and a known PIN is not confined to the tier the conditions actually imply. They assert whichever tier is cheapest to satisfy for the access they want.

This paper reports what happened when a tiered MFA capability was built this way and then measured. A SysML model governed the work, and three operating tiers were derived from one universal requirement set. The prototype is a Django web application with a Node-RED hardware orchestrator on a Raspberry Pi 5, using RFID, fingerprint, and PIN factors. An experiment harness runs an adversary holding a cloned RFID credential and a known PIN against twenty protected resources, with a legitimate subject holding all three factors as a control. The harness and the results it produces are in a public repository at a fixed commit, so every number reported here can be reproduced.

This paper makes three contributions. First, it defines a tier as a joint constraint over authentication factors, authorization scope, operating precondition, and accountability, rather than as a factor set. Second, it shows by measurement that where two tiers authorize the same scope, a factor required by only one of them protects nothing, and the adversary reached every resource the legitimate subject reached. Third, it identifies a composition defect in tiered degraded-access design, where a minimum assurance requirement and approval for degraded access cannot both be satisfied on the resources that most need both.

This paper asks three questions. First, when two operating tiers share authentication factors, what actually separates them? Second, what security properties hold, and fail to hold, when the operating tier is asserted by the requester rather than derived by the capability? Third, which defects did the governing model surface that the implementation and its own test suite did not? Section IV answers all three against a working prototype.

II. Background

Model-Based Systems Engineering (MBSE) is relevant here because this project spans hardware and software, and SysML supports requirements, behaviors, structures, and allocations [1]. Rapid prototyping surfaces implementation and interface issues early [2], [3]. Here MBSE governs the prototyping by connecting context, requirements, functions, logical architecture, and physical realization.

The views expressed in this paper are those of the authors and do not reflect the official policy or position of the United States Military Academy, the Department of the Army, or the Department of Defense.

A. MFA, Zero Trust, and Related Work

Multi-factor authentication combines knowledge, inherence, and possession factors to establish identity [5]. Zero Trust emphasizes explicit verification and policy-based access decisions, which motivates modeling authentication and authorization as separate responsibilities [6], [7]; this project is Zero Trust-informed and is not presented as compliant. Prior work extends SysML with security-aware constructs for industrial control systems [8]. This work does not extend the language; it reports what an ordinary SysML model surfaced about a system built against it. Emergency-access and break-glass models permit relaxed access and compensate with strengthened audit [9], [10].

B. Threat Model

The MFA capability is intended for cyber-physical systems in tactical settings, where degraded conditions are adversary-induced rather than random faults. Jamming, electronic-warfare denial, and loss of connectivity are the denied, degraded, intermittent, and limited (DDIL) conditions tactical systems must keep operating through, and they apply to the capability's own dependencies as much as to the mission it protects. The audit path in particular is a local or intermittently connected service, so it is subject to the same interference as everything else.

The adversary can both induce a degraded condition and present credentials they may have previously obtained. Knowledge and possession factors, such as a cloned badge or a known PIN, are assumed more readily obtained than inherence. A badge may be cloned from a brief read and a PIN may be observed as it is entered, while defeating the fingerprint requires a copy of the enrolled print or tampering with the sensor. Finding 2 does not depend on this, since that fingerprint is bypassed rather than defeated. The adversary who cannot satisfy the strongest configuration therefore has two openings rather than one. The first is a downgrade, where degradation forces the system into a weaker configuration the adversary can satisfy [11]. The second is that degradation is also what prompts a tier to be asserted at all, and the capability takes the asserted tier as an input rather than deriving it, so the adversary is not confined to the tier the conditions actually imply.

This capability does not control tier selection, and the prototype does not host the physical CPS itself. What it does is evaluate each access request against the asserted tier and record the result, and Section IV reports where that evaluation and that record did not hold. This framing follows the assume a hostile environment tenet of the DoD Zero Trust Reference Architecture, which treats the operating environment as adversarial by default rather than as a set of conditions that happen to fail [12].

III. Approach: MBSE-Guided Rapid Prototyping

A. MBSE-Guided Rapid Prototyping Method

In the first phase, Govern the model, the project defines the system context using a domain block definition diagram (BDD), establishes the model package structure and terminology, captures use cases and the system boundary, and defines universal requirements [1]. In the second phase, Derive structure, it builds the functional hierarchy, activities, and sequences, derives tier-specific requirements, and develops the logical architecture for each tier. In the third phase, Couple and refine, it maps logical elements to the physical prototype, implements the prototype informed by the model, and refines the model and the prototype in parallel based on implementation feedback.

B. Functional Hierarchy and Behavioral Modeling

A functional hierarchy organized the activity diagrams and decomposed the capability into logical components [1]. Detailed activities refined selected functions, such as factor collection and authentication evidence verification, bridging what the capability should do and how it was implemented.

C. Logical Architecture Modeling

The logical architecture is where the model went from abstraction to a constrained implementation [1]. Rather than prescribing hardware or software, it first defined logical responsibilities. Fig. 1 shows that decomposition for Tier 1, the baseline for comparing the other two. Node-RED then took factor collection and local hardware, while Django took credential management, authentication, and authorization, so the model-to-prototype traceability reflects logical architecture rather than coding decisions.

D. Case Study and Prototype Implementation

MFA was selected because it spans policy, physical sensors, and access decisions, and each tier expresses the same access-control intent through a different logical configuration. MFA was modeled as system-level access control, with factor collection, credential enrollment, authentication-evidence verification, authorization of resource access, and audit/accountability each modeled as a separate responsibility [5], [6].

E. Tier Definitions

The tier capabilities are defined in Table I.

They are not a simple strongest-to-weakest ranking; they are architecturally different responses to different operating conditions. Tier 1 is the fullest normal-operation capability, Tier 2 reduces hardware complexity while retaining controlled access and accountability, and Tier 3 emphasizes mission continuity when systems are degraded but access should not be denied outright, following the graceful-degradation pattern [13]. The prototype implemented modeled elements on a Raspberry Pi with a Django application and a Node-RED hardware integration layer. Django is the application and final decision maker; Node-RED is the local hardware orchestrator.

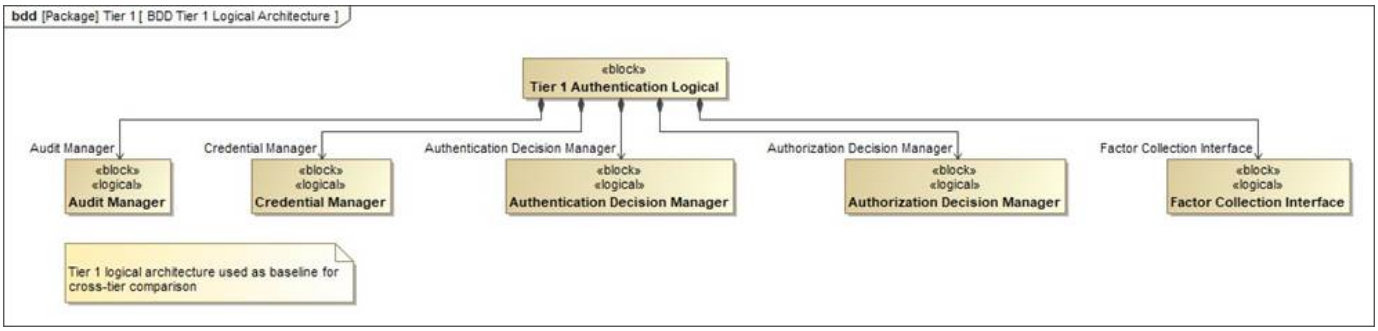


Fig. 1. Tier 1 authentication logical architecture decomposition BDD.

TABLE I
TIER DEFINITIONS.

Tier	Operating condition	Required factors	Access scope	Audit/accountability
Tier 1	Highest-complexity normal operation	RFID + fingerprint	Standard protected access	Strict audit
Tier 2	Simplified normal protected access	RFID + PIN	Standard protected access	Strict audit
Tier 3	Degraded mission-continuity access	RFID + PIN	Only resources approved for degraded access	Strict audit; deferred transmission permitted

F. Node-RED Hardware Layer and End-to-End Access Flow

Node-RED called the RFID and fingerprint scripts and exposed local HTTP endpoints to Django, connecting the cyber and physical domains. Django creates an authentication session, requests factors from Node-RED, evaluates authentication and authorization, makes the access decision, and records it in an audit event.

IV. MODEL-TO-PROTOTYPE TRACEABILITY AND FINDINGS

Maintaining correspondence between the model and the prototype is what made the tier definitions described below visible and attributable to specific model elements. Three traces carry the findings that follow. Authorization of resource access was realized in the Django service layer as _evaluate_authorization, Tier 3 degraded access was bound to the resource allow_degraded_access flag, and the Audit Manager logical element was realized as the Django AuditEvent model.

Developing the prototype against the model produced six findings: one concerning what separates one tier from another, four defects that were only visible once that separation was settled, two of which were fixed and measured and two of which are disclosed here and remain open, and one concerning why the tiers were comparable at all.

A. Experiment Setup

The numbers in this section come from run_experiments.py in the public repository at https://github.com/noahstephenson/MFA-System, which seeds a fixture into an isolated database and drives the same service-layer path a normal access request takes, mocking only the Node-RED hardware boundary. The results file it writes is in the repository as well.

The fixture is twenty protected resources, five of them approved for degraded access, and two enrolled subjects. The adversary holds a cloned RFID credential and a known PIN but has no enrolled fingerprint, which is the possession-and-knowledge attacker from the threat model. The legitimate subject holds all three factors. A resource is reached when the capability returns and persists an approved authorization decision for it. Protected resources here are database objects rather than operational endpoints, so this measures the authorization decision and not enforcement downstream of it.

Three things were varied: which tier the requester asserts, how many resources carry a minimum-assurance requirement, and whether the audit sink is reachable. The harness does not grant the adversary the ability to assert a tier. The prototype accepts any of the three defined tiers, and that unverified acceptance is what the experiment measures. The failure injected is a failure to write the local durable record, not loss of an external audit service, which the model permits to be queued for deferred transmission. Audit failure is induced after the session has already started, because a sink that is down from the outset blocks session creation, so only a mid-flow failure reproduces the behavior Finding 4 describes. Table II reports the results.

B. Finding 1: A tier is defined by more than its authentication factor set.

While building the prototype and the model, separating Tier 2 and Tier 3 in the physical implementation was by far the hardest part of the build, since both use RFID and PIN. At the authentication layer, they were effectively the same tier. Therefore, the separation was clearly not the factors. It was instead Tier 3's requirement to bind access to an approved subset of functions and a degraded operating condition.

TABLE II
CONTAINMENT AND ACCOUNTABILITY ACROSS SCENARIOS. UNAUDITED APPROVALS MEASURED UNDER MID-FLOW AUDIT FAILURE.

Scenario	Asserted tier	Reached of 20	Unaudited approvals, as built	With record enforced
Minimum assurance absent	Tier 2	20 / 20	20	0
Minimum assurance, 10 of 20 covered	Tier 2	10 / 20	10	0
Minimum assurance, 20 of 20 covered	Tier 2	0 / 20	n/a	n/a
Degraded scope (Tier 3 allowlist)	Tier 3	5 / 20	5	0
Control, legitimate subject, 10 of 20 covered	Tier 1	20 / 20	20	0
Any scenario, sink unavailable, record enforced	All three	0 / 20	n/a	0

Tier 3 authenticates "for degraded access to an approved subset of protected cyber-physical system functions" and denies access "when the requested function is not approved for degraded operation." The scope check is a separate condition from the factor check, so denial can follow either from failed factors or from an unapproved function, which are two independent conditions within one tier. This is built into the prototype the same way: authorization is evaluated separately from authentication, and the degraded-access check runs after the factor check because it is not part of authentication.

The word tier implies a single ordering from strongest to weakest, but no single ordering holds across these design points. Tier 1 and Tier 2 are both two-factor, and Tier 2 and Tier 3 use identical factors. NIST authentication assurance levels cannot distinguish all three tiers because they characterize the strength of the authentication event rather than the scope that event authorizes.

In this model, a tier is therefore a joint constraint over authentication factors, authorization scope, operating preconditions, and accountability behavior, and two tiers are distinct whenever they differ in any one of these. Defining a tier this way is motivated by the Structured Decomposition and Composition principle in NIST SP 800-160 Vol. 1 Rev. 1 [14], which treats separating security responsibilities into their own elements as a basis for engineering trustworthy systems.

C. Finding 2: Where two tiers authorize the same scope, a factor required by only one of them protects nothing.

This finding is the reverse of Finding 1: two tiers with different factors that turn out not to differ in the way that matters. Tier 1 requires RFID and fingerprint and Tier 2 requires RFID and PIN, but Table I gives both of them standard protected access. The factor sets are not the same, and the scope is. The adversary described in the threat model holds a cloned badge and a known PIN and cannot satisfy Tier 1, but there is no reason for that adversary to try. Asserting Tier 2 reaches everything a Tier 1 subject reaches.

Stated more generally, when the requester asserts the operating tier and more than one tier authorizes a given resource, the effective assurance of that resource is whatever the weakest of those tiers requires. The corollary is the part that matters here. An inherence factor that appears in only one of two scope-sharing tiers protects zero resources. The fingerprint in this capability was not a weaker control than intended. It was not a control.

This is adjacent to results that are already established: least privilege makes a similar point about scope, and choosing an assurance level based on what is being protected is standard practice. The narrower claim here is that the invariant applies wherever the operating tier is asserted by the requester rather than derived. NIST assurance levels characterize the authentication event and the session it establishes [5], and nothing in them orders operator-asserted modes or forbids two of them from authorizing identical scope.

The prototype makes the shared scope measurable. Against twenty protected resources at the default configuration, an adversary holding RFID and PIN with no enrolled fingerprint reached all twenty while asserting Tier 2, and the legitimate subject holding all three factors reached the same twenty while asserting Tier 1, in the control run where ten of the twenty required Tier 1. The ratio of Tier 2's authorized scope to Tier 1's is 1.00.

This is a finding about the model rather than about the code because there was nothing to check. The model gave a protected resource no way to state what assurance it required, so the prototype had no field to set and no condition to evaluate, and both tiers passed every resource because no resource had an opinion about tiers at all. Adding that property, and then raising it on specific resources, contains the adversary: resources reached fall from twenty of twenty at no coverage to zero of twenty at full coverage. The mechanism only supplies the ability to say it. Deciding which resources warrant which assurance is a policy decision that the method surfaced as necessary and did not answer on its own.

D. Finding 3: Minimum assurance above Tier 3 and degraded access cannot both be satisfied.

Once assurance is expressible per resource, a resource carries two properties that were specified independently of each other. One is the minimum tier permitted to reach it. The other is whether it is approved for access under degraded operation. Each of these is reasonable by itself, and together they cannot both be satisfied. The minimum-tier mechanism ranks Tier 3 below Tiers 1 and 2, so a resource requiring assurance above Tier 3 is unreachable in Tier 3 no matter what its degraded-access flag says. A resource cannot be both high-assurance and reachable when degraded, and the flag on any resource in that state is dead configuration that reads as though it does something.

This is backwards with respect to what the tiering is for: the resources most likely to be mission-essential under degradation are the same ones anyone would hold at high assurance, so the properties collide precisely where the capability should be most useful.

The constraint surfaced through the experiment design rather than through inspection of the specification. The set of resources carrying a minimum assurance requirement and the set approved for degraded access had to be built disjoint, because overlapping them drives Tier 3 to zero reachable resources by construction and the degraded scenario returns nothing. Having to design the fixture around the constraint is what made it visible. This is the same defect class as Finding 2, two tier properties specified separately and composed without any stated invariant relating them, and unlike Finding 2 it is unresolved in this build.

E. Finding 4: The model's accountability requirement held where the implementation and its tests did not.

Tier 3 reduces hardware complexity, service assumptions, and access scope, but the model does not reduce accountability: the universal accountability requirement covers attempts, outcomes, and access decisions across all three tiers, and no tier-specific requirement narrows it for Tier 3.

Holding that requirement over the local record, and not only over transmission to an external service, means access must be denied where the record cannot be written. Break-glass and optimistic-security models strengthen audit precisely because access was loosened [9], [10]; this model inverts that, holding accountability fixed and letting access give way. The prototype did not.

When the audit sink failed after a session had already started, all three tiers left approved access decisions in the database with no record of them, and they did it two different ways. Tiers 1 and 2 saved the decision first and then raised on the audit write, which returned an error to the caller while the approval stayed durable, leaving twenty and ten unaudited approved sessions. Tier 3 swallowed the failure entirely and left five. Making the decision record a precondition for approval brings the count to zero in all three tiers, but the guarantee that results is narrower than the requirement that motivated it: no access decision persists as approved without a durable record of that decision. Denials, session-start records, and the other audit events are still best-effort and are still lost when the sink is down.

The test suite passed the entire time, because it asserted the fail-open behavior as the expected result: a Tier 3 access attempt still succeeded when the audit write failed. The only artifact in the project that disagreed was the universal accountability requirement in the model.

Enforcing accountability this way is a trade rather than a straight improvement. With the decision record required and the audit sink unavailable, nobody reached anything in any scenario, including the legitimate Tier 1 subject holding all three factors. This means an adversary who can reach the audit path can deny access to everyone without touching authentication at all. That tension is not new, and prior work on accountable authentication treats denial of service by the log as a known problem [15]. What changes in a tactical setting is which side is cheaper to attack. The audit path here is a local or intermittently connected dependency rather than a cloud service, so denying accountability becomes a practical way to deny the mission continuity that the tiering exists to preserve.

F. Finding 5: The operating tier is an input to the capability, not an output of it.

Tier selection sits outside the system boundary: it is a decision made by the host CPS. In the model, tier is a property of the access policy; in the prototype, it arrives with the access request, so the capability evaluates against the asserted tier rather than selecting it.

This means the asserted tier is a security-relevant input. The capability cannot assume that a tier was selected honestly or correctly, and its security properties must hold regardless of which tier is asserted. The source and integrity of the operating-tier input must therefore be protected by the host CPS or validated at the system boundary.

In a contested environment, an externally detected degraded condition such as communications denial or jamming may change the operating tier without any authentication factor being presented, which is the downgrade case described in the threat model. The prototype does not bound this. The asserted tier is checked only for being one of the three defined tiers, and nothing compares it against any observed or reported condition, so inducing a degraded condition does not restrict an adversary to the tier that condition implies. Asserting Tier 3 would restrict access to the degraded subset, but no adversary would: Tier 2 takes the same two factors and returns the full scope. Degradation gives the adversary the opening to assert a tier; it does not constrain which one. Closing this would mean binding the assertion to a condition the capability derives or attests independently, and neither the model nor the prototype specifies that. It is disclosed here as an open defect and carried to future work.

G. Finding 6: Model governance and structured decomposition are what made the tiers comparable.

Model governance is what made the three tiers comparable. A shared glossary and package structure named tier-scoped elements consistently, deriving each tier's requirements from one universal set made them comparable as sets rather than as three separate lists, and the logical BDDs and internal block diagrams fixed responsibility allocation for all three tiers before any hardware was assigned. Separating authentication and authorization as distinct responsibilities is what made the differences in Findings 1 and 5 observable.

V. Limitations and Future Work

A. Limitations

The prototype is evidence of a possible implementation, not proof of security or compliance: protected resources were

text objects, not operational systems; it was not production-ready [4]; and it received no formal cybersecurity testing, being Zero Trust-informed but not evaluated against a defined baseline [6], [7]. Degraded operation was induced by configuration rather than by physically denying a sensor or a link, so the results describe how the authorization logic behaves under degraded conditions and not the physical conditions that would produce them. The fixture also required the resources carrying a minimum assurance requirement and the resources approved for degraded access to be disjoint sets, which is the constraint described in Finding 3, so the degraded scenario measures one side of a composition the capability cannot currently satisfy on both. The control confirms that legitimate Tier 1 access survives with ten of twenty resources covered, but access at full coverage was not measured. Lastly, these three tiers are one design configuration; other systems would draw different comparisons. However, Findings 2 and 3 are not specific to three tiers. With any number of tiers, effective assurance is still set by the weakest tier authorizing a resource, and a minimum above every degraded tier still removes degraded access. Both follow from the definitions and were measured only at three.

B. Future Work

One direction is guarded tier transitions with reauthentication to return to higher tiers, rather than conditions changing tiers on their own. Transitions would also need to be recorded as their own events, since a change in operating tier is a security-relevant decision in itself. Another would be binding the assertion to a condition the capability derives or attests independently, which is the open defect carried here from Finding 5. Verification should also be strengthened by deriving test cases through the model.

VI. Conclusion

The structure here is not specific to MFA. Any capability with more than one operating mode has the same problem: if the modes differ only in authentication factors, they are not fully specified, because factors describe the authentication event but not the scope and conditions it operates in. These design points apply to any mode-based access control in CPS.

Three operating tiers were derived from one universal requirement set, implemented in a Django and Node-RED prototype on a Raspberry Pi 5, and measured against an adversary holding a cloned badge and a known PIN. Where the tier is asserted by the requester, the effective assurance of a resource is whatever the weakest tier authorizing it requires: the adversary asserted Tier 2 and reached all twenty resources the legitimate Tier 1 subject reached, and nothing in the prototype objected. Nothing checks the asserted tier against the condition that prompted it, so degradation gives an adversary the opening to assert a tier without constraining which one. The defect the model caught on its own was accountability: under a mid-flow audit failure all three tiers left approved access decisions with no record of them, the test suite asserted that behavior as expected, and the model's universal accountability requirement was the only artifact that disagreed. Minimum assurance and degraded access remain unsatisfiable together on the resources that need both, and that defect is open in this build.

Acknowledgment

The authors used Anthropic's Claude for manuscript revision and prototype code. All design decisions, results, and conclusions are the authors' own, and the authors reviewed all text and code and take full responsibility for this paper.

References

[1] S. Friedenthal, A. Moore, and R. Steiner, *A Practical Guide to SysML: The Systems Modeling Language*, 3rd ed. Burlington, MA, USA: Morgan Kaufmann, 2015.

[2] A. M. Madni, "MBSE testbed for rapid, cost-effective prototyping and evaluation of system modeling approaches," *Applied Sciences*, vol. 11, no. 5, art. 2321, 2021.

[3] M. Meissner, G. Jacobs, P. Jagla, and J. Sprehe, "Model based systems engineering as enabler for rapid engineering change management," *Procedia CIRP*, vol. 100, pp. 61–66, 2021.

[4] J. Cederbladh, A. Cicchetti, and J. Suryadevara, "Early validation and verification of system behaviour in model-based systems engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 3, art. 81, 2024.

[5] D. Temoshok et al., "Digital identity guidelines: Authentication and authenticator management," NIST, Gaithersburg, MD, USA, NIST SP 800-63B-4, 2025.

[6] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," NIST, Gaithersburg, MD, USA, NIST SP 800-207, 2020.

[7] R. Chandramouli and Z. Butcher, "A zero trust architecture model for access control in cloud-native applications in multi-location environments," NIST, Gaithersburg, MD, USA, NIST SP 800-207A, 2023.

[8] R. Oates, F. Thom, and G. Herries, "Security-aware, model-based systems engineering with SysML," in *Proc. 1st Int. Symp. ICS & SCADA Cyber Security Research*, 2013, pp. 78–87.

[9] D. Povey, "Optimistic security: A new access control paradigm," in *Proc. Workshop New Security Paradigms (NSPW '99)*, 1999, pp. 40–45.

[10] A. Ferreira et al., "How to securely break into RBAC: The BTG-RBAC model," in *Proc. 25th Annu. Computer Security Applications Conf. (ACSAC)*, 2009, pp. 23–31.

[11] E. S. Alashwali and K. Rasmussen, "What's in a downgrade? A taxonomy of downgrade attacks in the TLS protocol and application protocols using TLS," in *Security and Privacy in Communication Networks (SecureComm 2018)*, LNICST vol. 255. Cham, Switzerland: Springer, 2018, pp. 468–487.

[12] Department of Defense, "Department of Defense (DoD) zero trust reference architecture, version 2.0," Defense Information Systems Agency & National Security Agency Zero Trust Engineering Team, 2022. [Online]. Available: https://dodcio.defense.gov/Portals/0/Documents/Library/(U)ZT_RA_v2.0(U)_Sep22.pdf

[13] S. Chu, J. Koe, D. Garlan, and E. Kang, "Integrating graceful degradation and recovery through requirement-driven adaptation," in *Proc. 19th Int. Symp. Software Engineering for Adaptive and Self-Managing Systems (SEAMS '24)*, 2024, pp. 122–132.

[14] R. Ross, M. Winstead, and M. McEvilley, "Engineering trustworthy secure systems," NIST, Gaithersburg, MD, USA, NIST SP 800-160 vol. 1, rev. 1, 2022.

[15] E. Dauterman, D. Lin, H. Corrigan-Gibbs, and D. Mazières, "Accountable authentication with privacy protection: The Larch system for universal login," in *Proc. 17th USENIX Symp. Operating Systems Design and Implementation (OSDI '23)*, 2023.